

High Order Functions

A Higher-Order Function is a function that either takes another function as an argument, returns a function, or both.

Common HOFs: map, filter, reduce, forEach, sort.

Example 1 — Function as an Argument

```
function greet(name) {  
  return `Hello ${name}`;  
}
```

```
function processUser(callback) {  
  console.log(callback("Prajwal"));  
}
```

```
processUser(greet);
```

Here, processUser is a **higher-order function** because it accepts another function (greet) as a parameter.

Example 2 — Function Returning Another Function

```
function multiplyBy(x) {  
  return function(y) {  
    return x * y;  
  };  
}
```

```
const double = multiplyBy(2);  
console.log(double(5)); // 10
```

multiplyBy is a higher-order function because it **returns a function**.

Q. Difference Between Higher-Order Function and Regular Function

“A regular function operates directly on values and returns a result, whereas a higher-order function operates on other functions—it can take functions as arguments or return them. Higher-order functions enable abstraction, reusability, and are fundamental to functional programming in JavaScript.

Q. Explain map vs forEach

map() is used to transform an array and returns a new array, making it ideal for functional programming patterns. forEach() simply iterates to perform side effects and does not return anything, so it's used when no transformation is needed

Q. Why HOFs are preferred in modern JS frameworks.

Higher-order functions are preferred in modern JavaScript frameworks because they let developers focus on *what they want to do* rather than *how to do it step by step*. Instead of writing repetitive loops and logic everywhere, we pass small functions that describe the behavior, and the framework handles the execution. This makes the code easier to reuse, easier to read, and less error-prone. It also helps manage things like state updates and async operations in a clean and predictable way

Functional Programming

Functional Programming is a way of writing code where we build programs using small, reusable functions that don't change data or depend on hidden state.

- ◇ Example (Normal vs Functional Thinking)

✗ Traditional (Imperative Style)

```
let result = [];  
for (let i = 0; i < numbers.length; i++) {  
  result.push(numbers[i] * 2);  
}
```

You control every step manually.

☑ Functional Style

```
const result = numbers.map(n => n * 2);
```

You describe **what you want**, not how to loop.

🔑 Core Ideas of Functional Programming

1 Pure Functions (No Side Effects)

Same input → Same output, always.

```
function add(a, b) {  
  return a + b;  
}
```

It doesn't modify anything outside.

2 Immutability (Don't Change Data)

Instead of modifying data, create new data.

```
// ✗ Changing original  
arr.push(4);  
  
// ☑ Creating new version  
const newArr = [...arr, 4];
```

3 Functions Are Treated Like Values

You can pass functions around just like numbers or strings.

```
const greet = name => `Hello ${name}`;
```

Why Functional Programming Is Popular Today

Functional programming helps:

- Write **cleaner code**
- Avoid unexpected bugs
- Make apps easier to test
- Handle async logic better
- Work well with modern tools like React